

AUDIT FRAMEWORK ANALYSIS

PracticePro Audit Framework Applicability

Mapping the complete web application audit framework to PracticePro (VEGA & Atrium), with Nigeria-specific compliance, security, and launch readiness assessment.

PLATFORM	PracticePro — VEGA & Atrium on Ati Gravity
STACK	React + Convex + Vite
MARKET	Nigeria (Legal Tech + PropTech)
DATE	May 2026

Table of Contents

1. Executive Summary	3
2. Audit Category Applicability Matrix	3
3. Performance Audits	4
3.1 Core Web Vitals	4
3.2 Bundle Analysis	4
4. Security Audits	5
4.1 Vulnerability Scanning	5
4.2 Broken Authentication Middleware (withFirmAuth)	5
4.3 Exposed API Keys	6
4.4 Emergency Offline Mode	6
4.5 OWASP Top 10 Compliance	6
5. Accessibility Audits (A11Y)	7
6. Code Quality Audits	8
6.1 Static Analysis	8
6.2 Technical Debt: Mock Data and Demo Paths	8
6.3 Schema Validation Disabled	9
7. Data Privacy and Compliance Audits	9
7.1 NDPA 2023 Compliance	9
7.2 Legal Industry Compliance (Nigerian Bar Association)	9
7.3 ISO 27001 and Landing Page Claims	10
8. API Audits	10
8.1 Convex Function Security	10
8.2 Rate Limiting	10
8.3 IDOR (Insecure Direct Object Reference)	11
9. Infrastructure Audits	11
10. User Experience Audits	11

11. Database Audits 12

 11.1 Data Integrity 12

 11.2 Backup and Recovery 12

12. Compliance Audit Logs 13

13. Nigeria-Specific Considerations 13

 13.1 Payment Integration Gaps 13

 13.2 Tax Compliance 13

 13.3 Tenant Vetting and Identity Verification 14

 13.4 WhatsApp Integration 14

14. Prioritized Launch Readiness Checklist 14

15. Recommended Audit Cadence for PracticePro 16

1. Executive Summary

This report maps a comprehensive 10-category web application audit framework to the specific context of PracticePro, a legal technology and property technology SaaS platform comprising the VEGA (legal practice management) and Atrium (property revenue monitoring) modules, built on the Ati Gravity platform using React and Convex. The platform targets the Nigerian market, serving law firms, property managers, and real estate portfolio operators. Each audit category has been evaluated against the actual codebase, with findings cross-referenced against a prior audit that identified 5 critical and 8 high-severity issues including exposed API keys, unauthenticated Convex functions, orphaned components, dual payment systems, and broken modal context management.

The analysis reveals that while PracticePro has implemented several commendable practices including DOMPurify sanitization, two-phase data hydration, manual Vite chunk splitting, and a tier-based feature gating system, significant gaps remain across security, compliance, accessibility, and code quality domains. The most urgent findings center on a fundamentally broken authentication middleware (withFirmAuth), exposed Gemini API keys in the frontend bundle, an emergency offline mode that grants Admin privileges without verification, and schema validation disabled in production. From a Nigerian regulatory standpoint, the platform claims NDPA 2023 compliance and ISO 27001 alignment on its landing page, but the underlying implementation falls short of both standards in several material respects, creating potential legal liability.

This document serves as both an applicability guide and a prioritized action plan. For each of the 10 audit categories, we assess relevance to PracticePro, identify current state versus target state, highlight Nigeria-specific considerations, and assign a priority classification using three tiers: [Must Fix] for issues that pose security, legal, or data integrity risks that must be resolved before launch; [Optional for V1] for improvements that enhance quality but are not launch-blocking; and [Remove for V1] for features or practices that should be stripped or disabled before the product reaches production users.

2. Audit Category Applicability Matrix

The following matrix provides a high-level overview of how each audit category in the framework applies to PracticePro. Each category is rated for relevance (Critical, High, Medium, Low) based on the nature of the application as a multi-tenant SaaS handling sensitive legal and financial data in a regulated Nigerian market. The current implementation status reflects what was found during the codebase analysis, not aspirational targets.

Audit Category	Relevance	Current Status	Launch Priority
1. Performance	High	Partial (chunking done, no Lighthouse)	[Must Fix]
2. Security	Critical	Broken auth middleware, exposed keys	[Must Fix]
3. Accessibility	Medium	Minimal (some aria-labels)	[Optional for V1]

4. Code Quality	High	150+ TODOs, no tests, schemaValidation:false	[Must Fix]
5. Data Privacy	Critical	NDPA claims unsubstantiated	[Must Fix]
6. API Audits	Critical	IDOR vulnerable, no rate limiting	[Must Fix]
7. Infrastructure	Medium	Convex-managed (partial coverage)	[Optional for V1]
8. UX Audits	Medium	No systematic testing	[Optional for V1]
9. Database	Critical	Dual payment systems, schemaValidation:false	[Must Fix]
10. Audit Logs	High	firmActivity exists, no formal audit log	[Must Fix]

Table 1: Audit Category Applicability Matrix for PracticePro

3. Performance Audits

3.1 Core Web Vitals

For a legal and property management SaaS where users frequently access the platform from offices in Lagos, Abuja, and Port Harcourt, often on inconsistent mobile broadband connections, Core Web Vitals are not merely a nice-to-have but a direct determinant of user retention. Nigerian mobile networks average 8-15 Mbps with latency spikes of 200-500ms, meaning that every millisecond of optimization translates directly into a better user experience for the target market.

PracticePro currently implements a two-phase data hydration strategy through `DataProvider.tsx`: Phase A fetches firm metadata (matters, contacts, properties, tasks, events, invoices) to unlock the UI within 500ms, while Phase B loads the full dataset (noteNotebooks, workflows, documentCategories) in the background. This is a well-architected pattern that should be preserved and measured. However, no Lighthouse or Core Web Vitals testing has been performed, so there is no baseline to measure against. The 12-provider context hierarchy (`AuthProvider`, `DataProvider`, `UIProvider`, `ProductProvider`, and 8 domain providers) may cause unnecessary re-renders, particularly when the `DataProvider` state updates trigger cascading re-renders across all child providers.

Finding: The `AuthProvider` implements a 20-second timeout with automatic retry for slow connections, with a comment referencing "Lagos mobile." This is appropriate for the target market but should be validated with real-world testing on Nigerian networks. Consider reducing the initial timeout to 12-15 seconds and implementing a more graceful loading skeleton rather than a splash screen hang.

3.2 Bundle Analysis

The Vite configuration in `vite.config.ts` implements manual chunk splitting via the `manualChunks` function, which is a strong practice. The current splits separate vendor-editor (`@tiptap/prosemirror`), vendor-convex, vendor-icons (`lucide-react`), vendor-pdf (`html2canvas/jspdf/pdfjs`), vendor-dates (`date-fns`), vendor-ai (`@google`), vendor-animation (`framer-motion`), and vendor-react. App-level splits separate

module-research, module-atrium, module-documents, and module-settings. This is an excellent starting point, but several concerns remain.

First, sourcemaps are enabled in production builds (sourcemap: true in vite.config.ts), which increases bundle size and potentially exposes source code structure. Second, the @tiptap ecosystem with 14 extensions is likely the single largest dependency, and lazy-loading it only when the DraftPro editor is accessed would significantly reduce the initial bundle. Third, framer-motion at 12.38.0 is a substantial animation library; replacing it with CSS transitions for simpler animations or using dynamic imports would reduce the vendor-animation chunk. Fourth, no vite-plugin-visualizer is configured, so there is no visibility into actual bundle composition or duplicate dependencies. Running a bundle analysis should be the first step before any optimization work begins.

Metric	Target	Current State	Action
LCP	< 2.5s	Unknown (no Lighthouse run)	[Must Fix] Run baseline Lighthouse audit
FID	< 100ms	Unknown	[Must Fix] Measure on Nigerian 3G/4G
CLS	< 0.1	Unknown	[Optional for V1] Test layout stability
Bundle Size	< 500KB gzip	Unknown (no visualizer)	[Must Fix] Install rollup-plugin-visualizer
Sourcemaps	Prod: disabled	Enabled in production	[Must Fix] Set sourcemap: false for prod

Table 2: Performance Audit Action Items

4. Security Audits

4.1 Vulnerability Scanning

Security is the single most critical audit category for PracticePro, given that the platform handles attorney-client privileged communications, litigation strategy documents, tenant financial records, and property ownership data. A breach would not only be a business disaster but could expose law firms to professional liability claims and violate the Nigerian Data Protection Act 2023 (NDPA). The codebase analysis reveals multiple severe vulnerabilities that must be addressed before any production launch.

4.2 Broken Authentication Middleware (withFirmAuth)

The withFirmAuth middleware in convex/lib/withAuth.ts represents the most critical security vulnerability in the entire codebase. This function is intended to enforce firm-level data isolation by validating that the caller belongs to the firm specified in the request. However, instead of calling ctx.auth.getUserIdentity() to

verify the authenticated user, it queries ALL users in the database (take(500)), finds any user with a matching firmId, and then allows the query to proceed even if no matching user is found. This means that any authenticated user can access any firm's data by simply providing the target firmId in their request. This is a textbook IDOR (Insecure Direct Object Reference) vulnerability that completely defeats multi-tenant data isolation.

Critical Vulnerability: withFirmAuth does NOT call getUserIdentity(). It queries all users, finds ANY user matching the firmId, and allows the request even if no user is found. This means any authenticated user can access any firm's data. Fix: Rewrite to use requireFirmUser() pattern from authHelpers.ts, which correctly calls ctx.auth.getUserIdentity() and matches by tokenIdentifier.

4.3 Exposed API Keys

The .env.example file documents three Gemini API key variables: API_KEY, GEMINI_API_KEY, and VITE_GEMINI_API_KEY. The VITE_ prefix exposes the key in the browser bundle, which is explicitly warned against in the .env.example comments. More critically, the vite.config.ts define block injects process.env.API_KEY and process.env.VITE_CONVEX_URL into the production bundle, making the Gemini API key accessible to anyone who inspects the JavaScript source. This is the same vulnerability identified in the previous audit (Issue C-1: Exposed API Keys) and remains unfixed.

The correct fix is to route all AI calls through Convex actions (server-side), which the codebase partially does through geminiService.ts and aiUtils.ts, but the VITE_GEMINI_API_KEY variable and the process.env.API_KEY injection in vite.config.ts must be removed entirely. The frontend should never have direct access to any API key; all AI operations should be proxied through Convex actions where the key is stored as a server-only environment variable.

4.4 Emergency Offline Mode

The AuthProvider in AuthContext.tsx implements an emergency offline mode that activates when the Convex backend is unreachable. While the intent is reasonable (maintaining usability during Nigerian network outages), the implementation creates a severe security vulnerability. When offline mode activates, the system creates a mock user object with the Admin role and isVerified: true, without any actual authentication. This means that anyone who can trigger a network failure (e.g., through DNS spoofing or a man-in-the-middle attack on a Lagos WiFi network) can gain full Admin access to the application. The offline user is assigned to "offline_firm" with Admin privileges, and while they cannot access real data (since there is no Convex connection), they can modify local state and potentially inject malicious data that syncs when connectivity is restored.

4.5 OWASP Top 10 Compliance

Mapping PracticePro against the OWASP Top 10 reveals deficiencies in at least 7 of the 10 categories. The most severe are Broken Access Control (A01:2021) due to withFirmAuth, Cryptographic Failures (A02:2021) due to exposed API keys, Insecure Design (A04:2021) due to the offline Admin bypass, Security Misconfiguration (A05:2021) due to sourcemaps and schemaValidation:false, and Identification

and Authentication Failures (A07:2021) due to the session management approach using email as the session token stored in localStorage. The password field exists in the user schema alongside mfaCode and verificationCode, suggesting that passwords may be stored in a reversible format rather than using bcrypt or Argon2, though the verifyLogin action mentions PBKDF2 hashing server-side.

OWASP Category	Status	PracticePro Finding	Priority
A01: Broken Access Control	FAIL	withFirmAuth does not verify identity	[Must Fix]
A02: Cryptographic Failures	FAIL	VITE_GEMINI_API_KEY exposed in bundle	[Must Fix]
A03: Injection	PASS	Convex handles SQL/NoSQL injection	N/A
A04: Insecure Design	FAIL	Offline mode grants Admin without auth	[Must Fix]
A05: Security Misconfiguration	FAIL	Sourcemaps, schemaValidation:false	[Must Fix]
A07: Auth Failures	PARTIAL	Email as session token in localStorage	[Must Fix]
A08: Data Integrity	PARTIAL	No file upload validation visible	[Optional for V1]
A09: Logging Failures	FAIL	No security event logging	[Must Fix]
A10: SSRF	UNKNOWN	External URL fetching not audited	[Optional for V1]

Table 3: OWASP Top 10 Compliance Assessment

5. Accessibility Audits (A11Y)

Accessibility for PracticePro operates on two levels: commercial and legal. From a commercial standpoint, government agencies and large corporate law firms in Nigeria increasingly require WCAG 2.1 Level AA compliance from their software vendors. The Nigerian Digital Economy Act and related policies are moving toward mandating accessibility for digital services. From a legal standpoint, the NDPA 2023 includes provisions about equal access to digital services, and while it does not yet explicitly mandate WCAG compliance, the trajectory of Nigerian digital regulation suggests this is coming. Proactive accessibility investment now avoids costly retrofitting later.

The current codebase shows minimal accessibility implementation. The LandingPage.tsx includes aria-label attributes on the theme toggle button and aria-hidden on decorative icons, which represents a basic awareness of accessibility but falls far short of WCAG 2.1 Level AA. There are no visible implementations of ARIA landmark regions, skip navigation links, focus trapping for modals, keyboard navigation support beyond browser defaults, or screen reader announcements for dynamic content updates. The

ModalManager.tsx system, which manages multiple overlapping modals, is particularly concerning because modal dialogs require focus trapping, escape key handling, and return focus on close to meet accessibility standards. The Tiptap editor (DraftPro) likely has some inherited accessibility from the ProseMirror framework, but custom extensions (LegalPlaceholder, LegalPartiesGroup, LegalContext) may break this if they do not implement proper ARIA attributes.

Given that V1 launch targets Nigerian law firms and property managers, and these users are primarily keyboard-centric professionals who value efficiency, basic keyboard navigation and focus management should be considered a [Must Fix] rather than optional. Full WCAG 2.1 AA compliance can be phased as [Optional for V1], but keyboard accessibility for critical flows (matter creation, rent collection, document signing) is non-negotiable for professional users.

6. Code Quality Audits

6.1 Static Analysis

The codebase contains over 150 TODO/FIXME/HACK/TEMP comments distributed across more than 100 files, with the heaviest concentrations in the DraftPro editor (61 TODOs), AloaChat (20 TODOs), SmartMatterModal (33 TODOs), and LandingPage (14 TODOs). While TODOs themselves are not inherently problematic, the density suggests significant incomplete implementation. More critically, the AloaChat.tsx and SaveToNoteForm.tsx files contain empty catch blocks (catch with no error handling), which silently swallow errors and make debugging extremely difficult. In a legal tech application, silent failures can lead to lost data, unrecorded actions, or incomplete audit trails.

The package.json includes an ESLint script ("lint": "eslint . --ext ts,tsx --report-unused-disable-directives --max-warnings 0"), but no .eslintrc configuration file was found in the codebase, suggesting that linting may not be actively enforced. TypeScript strict mode status is also unclear. No test framework (Jest, Vitest, Playwright) is configured in the dependencies, meaning there are zero automated tests. For a platform handling legal and financial data, the absence of any testing infrastructure is a significant risk that should be addressed before launch.

6.2 Technical Debt: Mock Data and Demo Paths

The codebase contains extensive demo and mock data infrastructure. The demoData.ts utility alone has 121 references to mock/demo/dummy data, and the AuthProvider includes a hardcoded demo user bypass (demo@practicepro.ng) that grants full Admin access. The DataProvider.tsx loads ATRIUM_DEMO_APP_STATE or VEGA_DEMO_APP_STATE when the demo user is detected, and the FloatingTestControls.tsx component (11 TODOs) appears to be a development-only testing overlay. While demo functionality is valuable for sales, the demo code paths are intertwined with production code rather than being isolated, creating risk of demo data leaking into production or demo bypasses being exploited.

Recommendation: Before launch, ensure demo mode is gated behind a feature flag or environment variable that is disabled in production builds. Remove FloatingTestControls.tsx from production builds entirely.

Audit all demo bypass paths to ensure they cannot be exploited to gain unauthorized access in production.

6.3 Schema Validation Disabled

The Convex schema in `schema.ts` is defined with `schemaValidation: false` (line 1026), which disables Convex's built-in runtime schema validation. This means that any document can be inserted into any table with arbitrary fields, and required fields can be omitted without error. Combined with the heavy use of `nullableString` and `v.optional(v.any())` throughout the schema, this creates a data integrity time bomb. In a legal application where data accuracy is paramount (court dates, filing deadlines, payment amounts), the absence of schema validation means that corrupt or incomplete data can silently enter the system without any error signal. This must be enabled before launch, even if it requires a data migration to fix existing records that do not conform to the schema.

7. Data Privacy and Compliance Audits

7.1 NDPA 2023 Compliance

The Nigerian Data Protection Act 2023 (NDPA) is the primary data protection regulation governing PracticePro. The landing page claims "NDPA 2023 Compliant" in both the trust badges strip and the footer, but the codebase analysis reveals significant gaps. While a `CookieConsent` component exists, it stores only a boolean acknowledgment (`practicepro_cookie_consent = "true"`) without granular consent categories (analytics, marketing, essential), which NDPA requires for lawful processing. There is no data export functionality (data portability right), no user-facing data deletion mechanism (right to be forgotten), and no data breach notification system. The `DataProcessingAgreement` component exists as a static document but is not backed by technical enforcement.

The NDPA also requires that personal data processing be limited to the purpose for which it was collected (purpose limitation). The current schema stores a wide range of personal data in the `users` table including `barNumber`, `dateOfBirth`, `nextOfKin`, and `identificationNumber`, but there is no visible mechanism for users to review, modify, or delete this data. The `deleteAccount` mutation exists in the `AuthContext` but its server-side implementation and whether it performs a complete deletion (including backups, related records, and analytics) has not been verified. For a platform claiming NDPA compliance, these gaps represent both regulatory risk and reputational damage potential.

7.2 Legal Industry Compliance (Nigerian Bar Association)

The Nigerian Bar Association (NBA) does not yet have formal technology certification requirements, but legal ethics rules impose obligations on lawyers regarding client confidentiality, which extends to the technology they use. The key requirements that PracticePro must satisfy include: client confidentiality (data encryption at rest and in transit), conflict checking before taking new matters (the matter creation flow should check for conflicts), document retention for 7+ years (currently no retention policy visible in the codebase), and audit trails for document access. The `firmActivity` table provides some audit logging, but it

does not track document-level access (who viewed which document when), which is essential for legal compliance. Additionally, the ExternalCounselInvite system creates temporary access for outside counsel, but there is no visible mechanism to enforce access expiration or audit what external counsel accessed during their window.

7.3 ISO 27001 and Landing Page Claims

The landing page footer states "ISO 27001 Aligned" and the trust badges strip displays "ISO 27001 Aligned" as a badge. This is a carefully chosen word: "aligned" rather than "certified." However, even the claim of alignment is questionable given the security vulnerabilities documented in this report. ISO 27001 requires, among other things, a risk assessment framework, access control policies, incident response procedures, and continuous monitoring. None of these are evidenced in the codebase. The claim "AES-256 at Rest" depends entirely on Convex's server-side encryption, which has not been independently verified. The claim "TLS 1.3 Encrypted" depends on the Convex and CDN configuration and may not be consistently enforced. These claims should either be substantiated with evidence or removed from the landing page before launch to avoid regulatory scrutiny under the NDPA and the Federal Competition and Consumer Protection Act (FCCPA), which prohibits misleading representations.

8. API Audits

8.1 Convex Function Security

PracticePro uses Convex as its backend, which provides built-in protection against SQL injection and NoSQL injection since all queries go through Convex's typed query API. However, Convex does not automatically enforce authorization; each query and mutation must explicitly check that the caller is authenticated and authorized. The codebase has two competing authorization patterns: `requireFirmUser()` in `authHelpers.ts`, which correctly calls `ctx.auth.getUserIdentity()` and validates the user's firm membership, and `withFirmAuth` in `withAuth.ts`, which does not. The inconsistency between these two patterns creates confusion for developers and increases the risk of new functions using the insecure pattern. All Convex functions should be audited to determine which authorization pattern they use, and `withFirmAuth` should either be fixed to call `getUserIdentity()` or deprecated in favor of `requireFirmUser()`.

8.2 Rate Limiting

There is no visible rate limiting on any Convex function or HTTP endpoint. For a SaaS platform that processes financial transactions (rent collection, invoice generation) and sends communications (WhatsApp reminders, emails), the absence of rate limiting creates vulnerability to abuse. A malicious user could potentially flood the system with rent payment records, generate thousands of invoices, or trigger mass WhatsApp notifications. Convex provides some built-in rate limiting through its usage tracking, but this operates at the account level, not per-user or per-function. Implementing application-level rate limiting for sensitive mutations (especially those involving payments and communications) should be a pre-launch requirement.

8.3 IDOR (Insecure Direct Object Reference)

Beyond the `withFirmAuth` vulnerability discussed in Section 4, the `checkResourceOwnership` function in `withAuth.ts` provides a proper IDOR check by comparing the resource's `firmId` with the caller's `firmId`. However, it is unclear whether this function is consistently used across all resource access patterns. The generic `baseActions` in `DataProvider.tsx` (`addItem`, `updateItem`, `deleteItem`) call Convex mutations that rely on the server-side function to enforce `firmId` isolation, but if those server-side functions use `withFirmAuth` instead of `requireFirmUser`, the IDOR protection is ineffective. A systematic audit of all Convex mutation and query functions is needed to verify that every function that accesses firm-scoped data uses the secure authorization pattern.

9. Infrastructure Audits

PracticePro runs on Convex as a managed backend, which significantly reduces the infrastructure attack surface. Convex handles database management, API endpoint provisioning, and basic security (TLS, DDoS protection at the infrastructure level). However, the platform still has infrastructure responsibilities that are not addressed. There is no visible backup strategy beyond Convex's default backups, and no backup restoration testing procedure. For a legal application where data loss could mean loss of court filings or client records, this is a significant gap. The NDPA requires data controllers to ensure the availability and resilience of processing systems, which implies both backup and disaster recovery capabilities.

The `convex/crons.ts` file exists, suggesting some scheduled operations, but the backup schedule and retention policy are not visible. SSL/TLS configuration depends on Convex and any CDN (likely Vercel based on the `vercel.json` file), and while these services typically provide strong TLS by default, the specific configuration has not been verified. The `.env.example` reveals Brevo (transactional email) and Chakra/Meta (WhatsApp) API integrations, both of which require their own security review. The Chakra integration is particularly sensitive because it involves sending messages on behalf of the firm, and compromised WhatsApp credentials could be used for social engineering attacks against tenants.

10. User Experience Audits

The previous audit identified several UX bugs including an orphaned `PropertyTrackingView` component, a discarded modal context, and an edit button that opens the wrong modal. These functional bugs are distinct from UX quality issues, which include inconsistent interaction patterns, unclear error states, and suboptimal information architecture. For the Nigerian market specifically, UX considerations must account for intermittent connectivity, variable screen sizes (the app should be tested on the Tecno, Infinix, and Samsung devices commonly used in Nigeria), and the expectation that professional software should "look the part" to command the subscription prices PracticePro charges.

The `FeatureGuard` component provides product-level gating (vega vs atrium), but the `tiers.ts` pricing system suggests that tier-level gating (Core vs Growth vs Pro vs Enterprise) is also needed. The current `FeatureGuard` only checks product type, not subscription tier, meaning that a Core-tier user on the Starter

plan could potentially access Pro-tier features if they navigate directly to the right URL. This is both a UX issue (users see features they cannot use) and a revenue protection issue (users may bypass paywalls). The Revenue Monitor tier-gating that was requested in the previous session should be implemented as part of a broader tier-aware FeatureGuard system.

11. Database Audits

11.1 Data Integrity

The Convex schema reveals several data integrity concerns. First, `schemaValidation: false` means that Convex will not reject documents that violate the defined schema, allowing corrupt data to enter the system silently. Second, the dual payment tracking systems create a consistency risk: the `ledger_entries` table tracks rent, service charges, penalties, and deposits with typed statuses (`pending`, `cleared`, `defaulted`), while the `properties` table has a `rentPaymentHistory` array field that tracks payments at the property level. These two systems are not synchronized, meaning that a payment recorded in one may not appear in the other, creating reconciliation difficulties for property managers who depend on accurate financial records. This was identified as a critical issue in the previous audit and remains unresolved.

Third, many fields use `nullableString` and `v.optional(v.any())` types, which provide minimal type safety. The `users` table stores sensitive fields (`password`, `mfaCode`, `verificationCode`, `failedLoginAttempts`, `lockedUntil`) alongside display fields, creating a risk that a query returning the full user document could leak credentials to the frontend. The `AuthContext` mitigates this with explicit field mapping (comment: "SECURITY: Explicit field mapping - NEVER spread raw data"), but the server-side `getUser` query in `myFunctions.ts` must also be audited to ensure it strips sensitive fields before returning data to the client. Fourth, there are no visible indexes on several frequently queried fields, such as `properties` by status or `contacts` by `contactType`, which could cause performance degradation as the dataset grows.

11.2 Backup and Recovery

Convex provides automatic backups as part of its managed service, but the backup retention period, restoration procedure, and point-in-time recovery capabilities have not been documented for PracticePro. For a legal application, the NBA and general legal practice standards require document retention for at least 7 years. While Convex likely provides sufficient backup for disaster recovery, the specific configuration needs to be verified and documented. Additionally, there is no visible mechanism for users to export their data (required by NDPA for data portability) or for administrators to perform point-in-time recovery of specific records (e.g., restoring a deleted matter). The `archive` table provides soft-delete functionality, but the `DataManagementSettings.tsx` component includes a "purge" function that permanently deletes all practice data, with no undo capability.

12. Compliance Audit Logs

The current `firmActivity` table provides basic activity logging (who did what, when, and on which matter/property), but it falls short of the structured audit logging required for legal compliance. The NDPA requires data controllers to maintain records of processing activities, and the NBA's ethical rules require law firms to maintain audit trails of document access. The audit framework template in the user's reference material provides an excellent starting point with typed actions (`MATTER_VIEWED`, `DOCUMENT_DELETED`, etc.), resource references, IP address tracking, and user agent logging. None of these are currently captured.

The `analytics_events` table captures some events but is focused on product analytics rather than compliance. What is needed is a dedicated `auditLogs` table with the following structure: `action` (typed enum), `resourceType`, `resourceId`, `userId`, `firmId`, `timestamp`, `ipAddress`, `userAgent`, `metadata` (optional). This table should be append-only (no updates or deletes allowed), should have automatic retention of at least 7 years, and should be queryable by regulators upon request. The Enterprise tier in `tiers.ts` promises "Audit Logs and SSO," so implementing this properly also fulfills a commercial commitment. The audit logging should cover: user login/logout, document access, data exports, permission changes, deletions (especially nuclear deletes), payment modifications, and sensitive configuration changes.

13. Nigeria-Specific Considerations

13.1 Payment Integration Gaps

The `tiers.ts` pricing is denominated in Naira, and the Atrium module processes rent payments, but there is no visible integration with Nigerian payment gateways such as Paystack or Flutterwave. The `PaymentGatewayModal.tsx` component exists with 3 TODOs, suggesting it is a placeholder. For the Revenue Monitor to deliver its core value proposition (automated rent collection), integration with at least one Nigerian payment gateway is essential. The current implementation appears to track payments manually (recording that a payment was received) rather than processing payments electronically. This gap is particularly significant because the Pro and Enterprise tiers promise "WhatsApp Automation Engine" and "Unlimited WhatsApp Reminders," which imply automated payment reminders and follow-ups, but without actual payment processing, the automation is limited to notifications rather than end-to-end rent collection.

13.2 Tax Compliance

The codebase includes a `NigerianTaxComplianceAgent.ts` in the agents directory and a `TaxDisclaimer.tsx` component in the reports section, indicating awareness of Nigerian tax requirements. The matters schema includes a `withholdingTaxApplicable` boolean field, and the invoices table includes `taxAmount` and `subTotal` fields. However, the specific tax calculations for Nigeria (7.5% VAT, withholding tax rates by service type, capital gains tax) are not visible in the codebase. The `TaxDisclaimer` component suggests that tax calculations may be presented with disclaimers rather than being computed. For a platform that generates invoices and processes payments, accurate tax computation is not optional; it is a legal

requirement under the Federal Inland Revenue Service (FIRS) regulations.

13.3 Tenant Vetting and Identity Verification

The Atrium Pro tier promises a "Tenant Vetting System," and the `leads_pipeline` table includes a `vettingScore` field and a "Vetted" stage. However, the codebase does not include integration with Nigerian identity verification services such as NIN (National Identification Number) verification or BVN (Bank Verification Number) lookup. The `AtriumPublicApplicationForm.tsx` component exists with 5 TODOs, suggesting it is partially implemented. For the tenant vetting system to deliver meaningful value, it must integrate with identity verification APIs. Several Nigerian providers offer these services (Smile Identity, Youverify, Prembly), and the integration should be prioritized for the Pro tier launch.

13.4 WhatsApp Integration

The Chakra/Meta WhatsApp API integration is a significant differentiator for PracticePro in the Nigerian market, where WhatsApp is the dominant communication channel. The `automation_logs` table tracks WhatsApp messages with typed categories (`rent_reminder`, `late_notice`, `payment_receipt`, `service_charge_alert`, etc.), and the `AutomationCenter.tsx` component provides a UI for managing automations. However, the `.env.example` shows `CHAKRA_` environment variables without indicating whether these are stored server-side only (no `VITE_` prefix, which is correct). The key risk is that WhatsApp API credentials, if exposed, could be used to send unauthorized messages on behalf of the firm, which would be a serious trust violation. Ensuring these credentials remain server-side only is critical.

14. Prioritized Launch Readiness Checklist

The following checklist organizes all findings from this audit framework analysis into three priority categories. Items marked [Must Fix] are launch-blocking issues that pose security, legal, or data integrity risks. Items marked [Optional for V1] improve quality but can be addressed post-launch. Items marked [Remove for V1] should be stripped from production builds before launch.

ID	Action Item	Domain
C-1	Rewrite <code>withFirmAuth</code> to call <code>getUserIdentity()</code>	Security
C-2	Remove <code>VITE_GEMINI_API_KEY</code> and <code>process.env.API_KEY</code> from <code>vite.config.ts</code>	Security
C-3	Gate offline mode: no Admin grants without auth verification	Security
C-4	Enable <code>schemaValidation:true</code> in Convex schema	Data Integrity
C-5	Synchronize <code>ledger_entries</code> with <code>rentPaymentHistory</code>	Data Integrity
C-6	Add rate limiting on payment and communication mutations	Security
C-7	Implement formal audit log table (append-only, 7yr retention)	Compliance

C-8	Remove or substantiate "ISO 27001 Aligned" and "NDPA 2023 Compliant" claims	Legal
C-9	Disable sourcemaps in production builds	Security
C-10	Implement tier-level FeatureGuard (not just product-level)	Revenue
C-11	Add NDPA-required granular cookie consent	Compliance
C-12	Verify getUser query strips password/mfaCode fields server-side	Security
C-13	Run baseline Lighthouse audit on Nigerian 3G/4G profile	Performance
C-14	Audit all Convex functions for secure auth pattern usage	Security

Table 4: [Must Fix] - Launch-Blocking Issues

ID	Action Item	Domain
O-1	Install rollup-plugin-visualizer for bundle analysis	Performance
O-2	Lazy-load DraftPro editor and Tiptap extensions	Performance
O-3	Add keyboard navigation and focus trapping for modals	A11Y
O-4	Implement WCAG 2.1 Level AA for critical user flows	A11Y
O-5	Set up Vitest + Playwright testing infrastructure	Quality
O-6	Integrate Paystack or Flutterwave for payment processing	Nigeria
O-7	Add NIN/BVN verification to tenant vetting system	Nigeria
O-8	Implement Nigerian tax calculations (VAT, WHT)	Nigeria
O-9	Add data export functionality for NDPA portability	Compliance
O-10	Implement cross-browser testing (Chrome, Safari, Edge)	UX
O-11	Add Hotjar or FullStory for UX session recordings	UX
O-12	Replace framer-motion with CSS transitions where possible	Performance
O-13	Document backup restoration testing procedure	Infrastructure
O-14	Run OWASP ZAP automated scan	Security

Table 5: [Optional for V1] - Post-Launch Improvements

ID	Item to Remove	Reason
R-1	FloatingTestControls.tsx (dev testing overlay)	Security
R-2	Sourcemap generation in production builds	Security

R-3	Unsubstantiated stats on landing page (10,000+, 5,000+, 1B+)	Legal
R-4	Demo Admin bypass in production builds (gate behind env var)	Security
R-5	Hardcoded mock data paths in production components	Quality
R-6	process.env polyfill injection in vite.config.ts define block	Security

Table 6: [Remove for V1] - Items to Strip from Production

15. Recommended Audit Cadence for PracticePro

Based on the specific risk profile of PracticePro as a Nigerian legal/proptech SaaS handling sensitive data, the following audit cadence is recommended. This cadence differs from the generic framework by emphasizing security and compliance audits more heavily than the typical SaaS, reflecting the regulatory environment and the sensitivity of legal and financial data. The cadence also accounts for the Nigerian context, where network reliability and device diversity add performance testing complexity.

Frequency	Audit Type	Tools/Methods
Daily/Automated	Dependency vulnerabilities, ESLint, Convex function auth check	npm audit, eslint, custom withFirmAuth scanner
Weekly	Lighthouse (Nigerian 3G/4G profile), bundle size, security headers	Lighthouse CLI, rollup-plugin-visualizer, SSL Labs
Monthly	OWASP ZAP scan, accessibility (Pa11y), NDPA compliance checklist	ZAP, Pa11y, manual NDPA review
Quarterly	Penetration testing, backup restoration test, UX review	External pentester, Convex backup restore, Hotjar analysis
Annually	External security audit, ISO 27001 gap analysis, NDPA audit	Certified auditor, ISO consultant, NDPA DPO review

Table 7: Recommended Audit Cadence for PracticePro

This cadence should be integrated into the Ati Gravity development workflow, with automated audits running as part of the CI/CD pipeline and manual audits scheduled as recurring calendar events. The weekly Lighthouse audit should use the "Slow 4G" throttling profile to simulate Nigerian network conditions, and the monthly NDPA checklist should be reviewed by a designated Data Protection Officer or compliance lead.